Правила сопоставления фактов с элементами модели

В таблицах 1-5 приведены предварительные правила сопоставления фактов, извлекаемых из артефактов программной системы, с элементами её архитектурной модели. Артефакты сгруппированы по видам: описание развёртывания, файлы сборки образов, спецификация API и исходный код на языке Python. Для каждого правила указаны: факт — часть артефакта, на который опирается правило; следствие — интерпретация факта; элемент модели — узел, связь или свойство архитектурной модели системы; уровень уверенности в полученном элементе. Описание правил в виде «условие — результат» опирается на подход MiSAR [1]. Набор правил и названия элементов будут уточняться в ходе разработки.

Элементы модели. Узел — сущность архитектуры (сервис, база данных, API). Связь — направленное отношение между двумя узлами, записывается «component1->component2 (ТИП)». Направление от компонента, который инициирует взаимодействие или владеет другим элементом. Свойство — характеристика узла.

Уровень уверенности. Явное — факт напрямую следует из артефакта. Выведенное — для получения элемента необходимо дополнительное уточнение: сопоставление имён, эвристика, подстановка значений переменных окружения.

Типы узлов. Application — приложение. Service — исполняемый компонент приложения, не являющийся базой данных или брокером сообщений. Database — хранилище данных. MessageBroker — брокер сообщений. Queue — очередь или топик брокера. API — программный интерфейс сервиса. Endpoint — операция API. ExternalSystem — внешняя система.

Типы связей. CONTAINS — приложение включает компонент. EXPOSES — сервис предоставляет API. HAS_ENDPOINT — API содержит операцию. CALLS — синхронный вызов. CONNECTS_TO — подключение к хранилищу или брокеру. PUBLISHES_TO, SUBSCRIBES_TO — отправка и получение сообщений. HOSTED_ON — очередь находится на брокере. STARTS_AFTER — порядок запуска компонентов.

Правила C5, P2-P6 основаны на этапе нормализации. Он выполняется после извлечения фактов и до формирования модели. Переменные окружения из разных файлов объединяются в общий набор. Адрес, используемый в коде, разрешается следующим образом «выражение в коде->переменная окружения->значение->имя хоста->компонент». Имя хоста сопоставляется с компонентом по имени сервиса или его алиасу, при несовпадении хост считается внешней системой. В путях параметры

заменяются на {}, из строк подключения сохраняются хост и имя переменной. Если адрес разрешить не удалось, связь не создаётся, а факт подлежит ручной валидации.

Таблица 1 — Уровень приложения

ID	Факт	Следствие	Элемент модели	Уверенность
S1	Компоненты описаны в одном docker-compose.yml	Считаем, что это одно приложение. Имя берётся из поля name, иначе из имени каталога	Узел Application; связь Application -> компонент (CONTAINS)	Выведенное

Таблица 2 — Docker Compose

ID	Факт	Следствие	Элемент модели	Уверенность
C1	build: ./services/holds	Образ собирается из исходного кода проекта	Узел Service {origin: own}	Явное
C2	image: postgres/mysql/mongo/ redis/rabbitmq/kafka	По имени образа определяем СУБД или брокер (допущение)	Узел Database{kind} или MessageBroker {kind}	Выведенное
C3	image не из словаря C2 и без build	Сторонний компонент	Узел Service {origin: external}	Выведенное
C4	depends_on	Сервис начинает работу после другого компонента	Связь component1->component2 (STARTS_AFTER)	Явное
C5	Значение в environment ссылается на имя компонента (URL, строка подключения)	Вероятно, сервис обращается к соответствующему компоненту	Связь service->component (CALLS — сервис, CONNECTS_TO — база данных/брокер сообщений)	Выведенное
C6	ports: “8080:8000“	Порт 8000 доступен с хоста на порту 8080	Свойство ports	Явное
C7	container_name, networks aliases	У компонента есть другие имена	Свойство aliases	Явное

Таблица 3 — Dockerfile

ID	Факт	Следствие	Элемент модели	Уверенность
D1	FROM	По базовому образу определяется язык	Свойство technology	Выведенное

Таблица 4 — OpenAPI

ID	Факт	Следствие	Элемент модели	Уверенность
O1	info в OpenAPI	Описание имени и версии API	Узел API	Явное
O2	Файл находится в каталоге сервиса или servers.url содержит соответствующий хост	API представляет данный сервис	Связь Service->API (EXPOSES)	Выведенное
O3	paths и методы	В API есть операция с данным методом и путём	Узел Endpoint; связь API->Endpoint (HAS_ENDPOINT)	Явное

Таблица 5 — Исходный код Python

ID	Факт	Следствие	Элемент модели	Уверенность
P1	Декораторы маршрутов FastAPI или Flask	Объявленный маршрут	Узел Endpoint (отсутствие OpenAPI иначе проверка)	Явное
P2	HTTP вызов	HTTP-запрос к компоненту	Связь Service->Endpoint (CALLS) при совпадении пути с эндпоинтом, иначе Service->Service	Выведенное
P3	подключение к БД	Подключение к хранилищу	Связь Service->Database (CONNECTS_TO)	Выведенное
P4	публикация сообщения	Отправка сообщения в очередь или топик	Узел Queue; связи Service->Queue (PUBLISHES_TO) и Queue->MessageBroker (HOSTED_ON)	Выведенное
P5	подписка на очередь	Сервис получает	Связь Service->Queue	Выведенное

		сообщения из данной очереди	(SUBSCRIBES_TO)	
P6	URL на хост вне приложения	Обращение к внешней системе	Узел ExternalSystem; связь Service->ExternalSystem (CALLS)	Выведенное

Список источников

1. Alshuqayran N., Ali N., Evans R. A model-driven architecture approach for recovering microservice architectures: Defining and evaluating MiSAR // Information and Software Technology. - 2025. - Vol. 186. - Art. 107808. - DOI: 10.1016/j.infsof.2025.107808.